

Réseaux pour le cloud

Jour 1 — Modèles, Ethernet et adressage IPv4

Bloc CL-RÉSEAU (jours 10-13 du cursus) — le calcul de sous-réseaux devient un réflexe

Objectifs de la journée

À la fin de cette journée, vous serez capable de :

- Expliquer **pourquoi le réseau est la compétence différenciante** de l'administrateur cloud.
- Situer un protocole dans les modèles **OSI** et **TCP/IP** et nommer les PDU (trame, paquet, segment).
- Décrire le rôle d'**Ethernet**, des adresses **MAC** et d'**ARP** sur un réseau local.
- Lire et écrire une adresse **IPv4** en notation **CIDR** (masque, adresse réseau, broadcast).
- **Calculer à la main** n'importe quel sous-réseau : c'est l'objectif n°1 de la journée.
- Reconnaître une adresse **IPv6** (notation seulement, pas de calcul).

Aujourd'hui est une journée d'**entraînement intensif** : deux séries d'exercices de calcul, corrections croisées, et un quiz final.

Plan de la journée

1. **Pourquoi le réseau ?** — la compétence qui fait la différence.
2. **Les modèles en couches** — OSI et TCP/IP, encapsulation.
3. **Ethernet, MAC, ARP** — ce qui se passe sur le réseau local.
4. **Adressage IPv4** — binaire, CIDR, masque, réseau, broadcast.
5. **Ateliers de subnetting** — deux séries d'exercices, la méthode posée.
6. **IPv6 en survol** — savoir lire une adresse, sans calculer.
7. Récap + quiz de fin de journée.

1. Pourquoi le réseau est LA compétence différenciante

Le cloud, c'est du réseau défini par logiciel

Dans le cloud, **tout ce que vous ne touchez pas physiquement, vous le décrivez** :

- Un VPC AWS ? C'est **un plan d'adressage** que vous écrivez (`10.0.0.0/16`).
- Un sous-réseau public ou privé ? C'est **une table de routage** que vous choisissez.
- Un security group ? C'est **une règle de pare-feu** que vous rédigez.
- Un load balancer ? C'est **de la répartition de charge L4/L7** que vous configurez.

Le cloud a supprimé les câbles et les baies, **pas les concepts**. Celui qui ne sait pas calculer un `/24` recopie des tutoriels ; celui qui maîtrise conçoit des architectures.

Fil conducteur du cursus : automatiser, superviser, sécuriser — et les trois commencent par comprendre ce qui circule et par où.

Où ce bloc vous emmène dans le cursus

Ce que vous apprenez cette semaine	Où vous le réutilisez
Calcul CIDR / sous-réseaux	CL-AWS1 J3 (VPC), CL-TP2 (plan d'adressage StockLine)
Routage, NAT	Tables de routage VPC, NAT Gateway
DNS	Route 53 (CL-AWS1 J7), certificats ACM
TLS, load balancing	ALB + HTTPS du TP2
Pare-feu stateful/stateless	Security groups vs NACL
Plan d'adressage cloud (J4)	Directement le TP2 : StockLine 3-tiers multi-AZ

Dès **CL-TP1** (dans 3 jours), vous documenterez le plan d'adressage de votre VM.

Au **CL-TP2**, vous construirez le VPC que vous allez concevoir **sur papier vendredi**.

2. Les modèles en couches

OSI et TCP/IP, sans par cœur inutile

Pourquoi découper en couches ?

Faire communiquer deux machines pose **des problèmes de natures différentes** :

- transporter des bits sur un câble ou une onde (physique) ;
- livrer au bon voisin sur le réseau local (liaison) ;
- traverser plusieurs réseaux jusqu'à la bonne machine (réseau) ;
- livrer à la bonne application, sans perte (transport) ;
- se comprendre entre applications (application).

Le principe : **chaque couche rend un service à celle du dessus** et s'appuie sur celle du dessous, sans connaître les détails des autres.

C'est exactement la logique que vous retrouverez dans le cloud : le security group travaille en couches 3-4, l'ALB peut travailler en couche 7.

Le modèle OSI – 7 couches

7	APPLICATION	HTTP, DNS, SSH	<- ce que voit l'appli
6	PRÉSENTATION	chiffrement, formats	(souvent fusionnées
5	SESSION	dialogues	avec la 7 en pratique)
4	TRANSPORT	TCP, UDP (ports)	<- SEGMENT
3	RÉSEAU	IP (adresses)	<- PAQUET
2	LIAISON	Ethernet (MAC)	<- TRAME
1	PHYSIQUE	câble, fibre, radio	<- bits

Moyen mnémotechnique (de bas en haut) :

« **Pour Le Réseau, Tout Se Passe Ainsi** »

Ce qu'il faut vraiment retenir d'OSI

En pratique, l'administrateur cloud utilise le modèle OSI comme **un vocabulaire commun** :

- « un équipement **de couche 2** » = il regarde les adresses **MAC** (switch) ;
- « un équipement **de couche 3** » = il regarde les adresses **IP** (routeur) ;
- « un filtre **de couche 4** » = il regarde les **ports** TCP/UDP (security group) ;
- « un load balancer **de couche 7** » = il lit le contenu **HTTP** (ALB).

Les couches 5 et 6 existent dans le modèle, mais dans la vraie vie elles sont **absorbées par la couche application**. Ne perdez pas de temps dessus.

À retenir absolument : **2 = MAC, 3 = IP, 4 = ports, 7 = application**.

Le modèle TCP/IP – celui du monde réel

OSI		TCP/IP	Exemples
+-----+		+-----+	
7 Application			HTTP, DNS,
6 Présentation	=>	APPLICATION	SSH, TLS
5 Session			
+-----+		+-----+	
4 Transport	=>	TRANSPORT	TCP, UDP
+-----+		+-----+	
3 Réseau	=>	INTERNET	IP, ICMP
+-----+		+-----+	
2 Liaison	=>	ACCÈS RÉSEAU	Ethernet,
1 Physique			Wi-Fi
+-----+		+-----+	

L'encapsulation : des poupées russes

```

Application :      [ données HTTP ]
Transport  :      [ TCP | données HTTP ]      = SEGMENT
Internet   :      [ IP | TCP | données HTTP ]  = PAQUET
Accès rés. : [Eth|IP |TCP | données HTTP |FCS] = TRAME
              |   |   |
              |   |   +-- ports source/destination
              |   +----- IP source/destination
              +----- MAC source/destination
    
```

À l'envoi on **encapsule** (on ajoute des en-têtes),
à la réception on **désencapsule** couche par couche.

Trame, paquet, segment : le bon mot au bon niveau

PDU (unité de données)	Couche	Contient	Adresse utilisée
Segment (ou datagramme UDP)	Transport (4)	données application	ports (ex. 443)
Paquet	Internet (3)	un segment	adresses IP
Trame	Accès réseau (2)	un paquet	adresses MAC

Points essentiels :

- Les adresses **IP ne changent pas** de bout en bout (hors NAT, vu demain).
- Les adresses **MAC changent à chaque saut** : chaque routeur ré-encapsule la trame pour le tronçon suivant.
- Quand un outil parle de « paquets » (tcpdump), il capture en réalité des **trames** et vous montre les couches au-dessus.

3. Ethernet, MAC et ARP

Ce qui se passe sur le réseau local

Ethernet et l'adresse MAC

Ethernet est le protocole de couche 2 dominant (câble et, dans l'esprit, Wi-Fi).

L'**adresse MAC** (Media Access Control) :

- **48 bits**, notée en hexadécimal : `52:54:00:8a:3b:1f` ;
- gravée dans la carte réseau (modifiable logiciellement) ;
- les 3 premiers octets identifient le **constructeur** (OUI) ;
- unique **sur le réseau local** — elle ne sert qu'entre voisins directs ;
- `ff:ff:ff:ff:ff:ff` = **broadcast** : tout le monde sur le segment reçoit.

Voir la sienne sur la VM :

```
ip link show          # ligne link/ether = adresse MAC
```

Le **switch** (commutateur) relie les machines du réseau local : il apprend quelle MAC est derrière quel port et livre chaque trame au bon destinataire.

Dans le cloud vous n'en verrez jamais (AWS le simule), mais le concept de « réseau local = domaine de broadcast » reste la base de ce qu'est un **sous-réseau**.

ARP : « qui a cette adresse IP ? »

Problème : pour envoyer un paquet IP à `192.168.1.20`, ma carte réseau doit construire une trame... avec **quelle adresse MAC de destination ?**

ARP (Address Resolution Protocol) fait le lien IP → MAC :

1. Broadcast : « **Who has** 192.168.1.20 ? Tell 192.168.1.10 » ;
2. La machine concernée répond : « 192.168.1.20 **is at** 52:54:00:8a:3b:1f » ;
3. La réponse est gardée en **cache ARP** quelques minutes.

```
ip neigh show          # afficher le cache ARP de la VM
```

Réflexe cloud : quand deux machines « du même réseau » ne se voient pas, vérifier le cache ARP fait partie du diagnostic de base.



Démo 1-1 — Observer le réseau de la VM

Fichier : `demos/04-cl-reseau/demo-1-1-observer-le-reseau.md`

Au programme, en direct sur la VM Ubuntu (outils vus en CL-LINUX J3) :

- `ip a` et `ip link` : interfaces, MAC, IP, notation `/24` déjà sous vos yeux ;
- `ip neigh` : le cache ARP avant/après un `ping` ;
- `tcpdump -i any arp` : **voir** un échange ARP en vrai ;
- `tcpdump -i any icmp` : suivre un ping et lire les couches.

Objectif : relier chaque ligne de sortie aux couches du modèle – plus rien de « magique » dans `ip a`.

4. Adressage IPv4

32 bits qui gouvernent tout le cloud

Une adresse IPv4 = 32 bits en 4 octets

192.168.10.77 est une écriture « humaine » de **32 bits** :

```
192      . 168      . 10      . 77
11000000 . 10101000 . 00001010 . 01001101
[octet 1] [octet 2] [octet 3] [octet 4]
```

- Chaque **octet** = 8 bits = une valeur de **0 à 255**.
- $2^{32} \approx 4,3$ milliards d'adresses au total – d'où la pénurie, le NAT et IPv6.
- Une adresse IP a **deux parties** : la partie **réseau** (où ?) et la partie **hôte** (qui, dans ce réseau ?). C'est le **masque** qui trace la frontière.

Toute la journée repose sur cette phrase : **le masque dit où s'arrête le réseau et où commence l'hôte.**

Binaire ↔ décimal : la méthode des poids

Chaque bit d'un octet a un **poids** :

```
poids : 128  64  32  16   8   4   2   1
bits  :  1   1   0   0   0   0   0   0  = 128+64 = 192
bits  :  1   0   1   0   1   0   0   0  = 128+32+8 = 168
bits  :  0   1   0   0   1   1   0   1  = 64+8+4+1 = 77
```

Décimal → binaire : on retrace les poids en partant de 128.

Exemple 203 : $203 - 128 = 75$, $75 - 64 = 11$, (32 non, 16 non), $11 - 8 = 3$, (4 non), $3 - 2 = 1$, $1 - 1 = 0$

→ **11001011**.

Apprenez **la ligne des poids par cœur** : **128 64 32 16 8 4 2 1**.

Vous l'écrirez en haut de chaque brouillon d'exercice.

Échauffement express (à l'oral, tous ensemble)

Convertissez de tête ou au brouillon – correction immédiate à l'oral :

Décimal → binaire	Binaire → décimal
255 → ?	10000000 → ?
0 → ?	11111111 → ?
128 → ?	11000000 → ?
192 → ?	11100000 → ?
224 → ?	11110000 → ?
240 → ?	11111110 → ?

Vous remarquez quelque chose ? Ces valeurs (128, 192, 224, 240, 248, 252, 254, 255)

sont **les seules possibles dans un masque** : des 1 contigus à gauche, puis des 0.

Retenez cette suite : elle revient dans tous les calculs.

Les classes A/B/C : de l'histoire ancienne (mais on vous en parlera)

Jusqu'en 1993, la partie réseau était fixée par le **premier octet** :

Classe	Plage du 1er octet	Masque implicite	Exemple
A	1 - 126	255.0.0.0 (/8)	10.0.0.0
B	128 - 191	255.255.0.0 (/16)	172.16.0.0
C	192 - 223	255.255.255.0 (/24)	192.168.1.0
D	224 - 239	multicast	—
E	240 - 255	expérimental	—

C'était **rigide et gaspilleur** → remplacé par le **CIDR** (Classless Inter-Domain Routing) : le masque devient explicite et libre.

À retenir : les classes ne servent plus **qu'à comprendre les vieux documents** et certains QCM de certification. Aujourd'hui, **tout est CIDR**.

Adresses privées et adresses spéciales

Certaines plages sont **réservées** (RFC 1918) : jamais routées sur Internet, librement utilisables en interne – **et dans vos VPC** :

Plage privée	CIDR	Couvre
10.0.0.0	/8	10.0.0.0 → 10.255.255.255
172.16.0.0	/12	172.16.0.0 → 172.31 .255.255
192.168.0.0	/16	192.168.0.0 → 192.168.255.255

Autres adresses à reconnaître :

- **127.0.0.1** (**loopback**, « moi-même », toute la plage 127.0.0.0/8) ;
- **169.254.0.0/16** (**APIPA/link-local** : « je n'ai pas eu de DHCP » – et chez AWS, **169.254.169.254** = service de métadonnées, vous le reverrez) ;
- **0.0.0.0/0** = « **tout Internet** » dans une règle ou une route.

Piège classique : **172.32.0.1** n'est **pas** privée (la plage s'arrête à 172.31).

Le masque de sous-réseau

Le **masque** est une suite de 32 bits : des **1** sur la partie **réseau**,
des **0** sur la partie **hôte**.

```
adresse : 192.168.10.77  11000000.10101000.00001010.01001101
masque  : 255.255.255.0  11111111.11111111.11111111.00000000
                        [----- réseau -----] [ hôte ]
```

Opération fondamentale : **adresse ET logique masque = adresse du réseau**.

Ici : **192.168.10.77** ET **255.255.255.0** → réseau **192.168.10.0**.

Deux machines sont « dans le même réseau » si l'opération donne

le même résultat des deux côtés. C'est ce calcul que fait votre machine
avant **chaque** envoi de paquet : voisin direct, ou passage par la passerelle ?

La notation CIDR : /n

Écrire `255.255.255.0` est long → on note simplement **le nombre de bits à 1** : `/24`.

`192.168.10.0/24` se lit : « le réseau 192.168.10.0, dont les **24 premiers bits** sont la partie réseau ». Il reste **32 – 24 = 8 bits** pour les hôtes.

CIDR	Masque	Bits hôte	Adresses	Hôtes utilisables
/24	255.255.255.0	8	256	254
/25	255.255.255.128	7	128	126
/26	255.255.255.192	6	64	62
/27	255.255.255.224	5	32	30
/28	255.255.255.240	4	16	14
/29	255.255.255.248	3	8	6
/30	255.255.255.252	2	4	2

Hôtes = $2^{(\text{bits hôte})} - 2$: la première adresse (réseau) et la dernière (broadcast) ne sont pas attribuables à une machine.

Adresse réseau, broadcast, plage utilisable

Dans un sous-réseau, quatre valeurs à savoir retrouver **en toutes circonstances** :

Valeur	Définition	Exemple avec 192.168.10.0/26
Adresse réseau	tous les bits hôte à 0	192.168.10.0
Première adresse utilisable	réseau + 1	192.168.10.1
Dernière adresse utilisable	broadcast – 1	192.168.10.62
Adresse de broadcast	tous les bits hôte à 1	192.168.10.63

L'adresse réseau **désigne** le sous-réseau (c'est elle qu'on écrit dans les tables de routage et les CIDR de VPC) ; le broadcast s'adresse à **tous** les hôtes du sous-réseau.

Anticipation cloud : AWS réserve en plus 3 adresses par sous-réseau (détail vendredi) – mais réseau et broadcast, c'est **universel**.

LA méthode en 4 étapes (à appliquer 25 fois aujourd'hui)

Pour toute question du type « **A.B.C.D/n** : réseau ? broadcast ? plage ? » :

1. **Octet intéressant** : celui où tombe la frontière du masque
(/1-/8 → 1er, /9-/16 → 2e, /17-/24 → 3e, /25-/32 → 4e).
2. **Taille de bloc** : **256 - valeur du masque dans cet octet**
(ex. masque 192 → bloc de 64 ; masque 240 → bloc de 16).
3. **Adresse réseau** : le multiple du bloc **inférieur ou égal** à la valeur de l'octet intéressant (les octets suivants passent à 0).
4. **Broadcast** : début du sous-réseau **suivant - 1**
(les octets suivants passent à 255).

Puis : première utilisable = réseau + 1 ; dernière = broadcast - 1.

Pas de binaire à poser à chaque fois : **le binaire justifie la méthode, la taille de bloc l'exécute.**

Exemple posé de bout en bout :

192.168.10.77/26

1. $/26 = 24 + 2 \rightarrow$ frontière dans le **4e octet**.
Masque : 255.255.255.192 (car $/26 \rightarrow 11000000 = 192$ dans le 4e octet).
2. Taille de bloc : $256 - 192 = 64 \rightarrow$ sous-réseaux : .0, .64, .128, .192.
3. 77 est entre 64 et 127 $\rightarrow$ **réseau = 192.168.10.64**.
4. Sous-réseau suivant : .128 $\rightarrow$ **broadcast = 192.168.10.127**.

Résultat complet :

Réseau	1re utilisable	Dernière	Broadcast	Hôtes
192.168.10.64/26	192.168.10.65	192.168.10.126	192.168.10.127	62

Vérification binaire (une fois, pour se convaincre) :

$77 = 01001101 \rightarrow$ bits réseau 01... hôte 001101 ; hôte à 0 $\rightarrow 01000000 = 64. \checkmark$

Le « tableau magique » à reconstruire de tête

À savoir **reconstruire en 30 secondes** en haut de votre brouillon :

Dernier octet du masque	128	192	224	240	248	252
CIDR (4e octet)	/25	/26	/27	/28	/29	/30
Taille de bloc	128	64	32	16	8	4
Hôtes utilisables	126	62	30	14	6	2

Le même tableau **se décale d'un octet** :

- masque dans le **3e octet** → /17 à /24, blocs de 128 à 1 **dans le 3e octet**
(ex. /20 → masque 255.255.240.0, blocs de 16 : 10.0.**0**.0, 10.0.**16**.0, 10.0.**32**.0...);
- règle universelle : **bloc = 256 - masque** dans l'octet intéressant.

Deuxième exemple posé : 172.16.50.10/20 (frontière au 3e octet)

1. /20 = 16 + 4 → frontière dans le **3e octet**. Masque : 255.255.240.0.
2. Bloc : 256 - 240 = 16 → sous-réseaux au 3e octet : 0, 16, 32, 48, 64...
3. Octet intéressant = 50 → multiple de 16 ≤ 50 : **48** → réseau = **172.16.48.0**.
4. Suivant : 64 → broadcast = **172.16.63.255** (octets à droite → 255).

Réseau	1re utilisable	Dernière	Broadcast	Hôtes
172.16.48.0/20	172.16.48.1	172.16.63.254	172.16.63.255	$2^{12} - 2 = 4094$

Les deux réflexes qui sauvent :

l'octet intéressant d'abord, **la taille de bloc** ensuite.

Tout le reste en découle.

Combien d'hôtes ? Quel préfixe choisir ?

Question inverse, omniprésente dans la conception de VPC :

« il me faut N machines, quel préfixe ? »

Méthode : chercher la **puissance de 2 strictement supérieure à N + 2**.

Besoin (hôtes)	N + 2	Puissance de 2	Bits hôte	Préfixe
2 (liaison)	4	4	2	/30
25	27	32	5	/27
60	62	64	6	/26
100	102	128	7	/25
500	502	512	9	/23
1000	1002	1024	10	/22

Bon réflexe d'architecte : **prévoir la croissance** — si le besoin est

« 100 aujourd'hui, le double demain », prenez /24, pas /25.

Découper un réseau en sous-réseaux

Découper, c'est **emprunter des bits à la partie hôte** :
chaque bit emprunté **double** le nombre de sous-réseaux et **divise par deux**
leur taille.

Exemple : découper **192.168.1.0/24** en **4 sous-réseaux** égaux.

1. 4 sous-réseaux = $2^2 \rightarrow$ **2 bits empruntés** $\rightarrow$ nouveaux préfixes **/26**.
2. Bloc : $256 - 192 =$ **64**.
3. On liste :

Sous-réseau	Plage utilisable	Broadcast
192.168.1.0/26	.1 $\rightarrow$.62	.63
192.168.1.64/26	.65 $\rightarrow$.126	.127
192.168.1.128/26	.129 $\rightarrow$.190	.191
192.168.1.192/26	.193 $\rightarrow$.254	.255

C'est **exactement** l'opération que vous ferez vendredi : VPC /16 $\rightarrow$ sous-réseaux /24.



Exercice 1-1 — Première série de calculs

(45 min)

Fichier : `exercices/04-cl-reseau/exercice-1-1-lire-le-cidr.md`

12 questions progressives, **à la main**, calculatrice interdite :

- conversions masque ↔ CIDR ;
- nombre d'adresses et d'hôtes ;
- adresse réseau / broadcast / plage pour des adresses données ;
- « ces deux machines sont-elles dans le même sous-réseau ? » ;
- reconnaître les plages privées.

Déroulement : **20 min seul**, puis **10 min de comparaison en binôme**

(vous devez être d'accord ET savoir expliquer), puis correction au tableau —
c'est vous qui posez les calculs.

Correction croisée : comment on pose un calcul au tableau

Format attendu à chaque question (et au TP1, et en entretien d'embauche) :

Question : réseau et broadcast de 10.20.135.90/21 ?

1. /21 -> 3e octet (16+5), masque 255.255.248.0
2. bloc = 256 - 248 = 8
3. 3e octet = 135 -> multiple de 8 <= 135 : 128
=> réseau = 10.20.128.0
4. suivant 136 => broadcast = 10.20.135.255
plage : 10.20.128.1 -> 10.20.135.254 ($2^{11} - 2 = 2046$ hôtes)

Une réponse **sans les étapes ne compte pas** : au TP noté, le raisonnement posé rapporte des points même si le résultat final est faux.



Exercice 1-2 – Deuxième série : découpage et VLSM (60 min)

Fichier : `exercices/04-cl-reseau/exercice-1-2-decouper-des-reseaux.md`

11 questions, difficulté croissante :

- découper un /24 et un /16 en N sous-réseaux égaux ;
- choisir le bon préfixe pour un besoin en hôtes ;
- **VLSM** : découpage en sous-réseaux de tailles différentes sans gaspillage ;
- agrégation : résumer deux /24 en un /23 ;
- pièges : adresse de broadcast déguisée en hôte, passerelle hors sous-réseau.

Même déroulement : seul → binôme → correction croisée au tableau.

La question 11 (VLSM complet) est le niveau attendu au TP1.

5. IPv6 en survol

Savoir lire, sans calculer

IPv6 : pourquoi, et à quoi ça ressemble

IPv4 : 2^{32} adresses — épuisées. IPv6 : **128 bits** → 2^{128} ($\approx 3,4 \times 10^{38}$).

Notation : **8 groupes de 16 bits en hexadécimal**, séparés par **:**

```
2001:0db8:0000:0000:0000:0000:0042:8329
```

Deux règles d'abréviation (à savoir lire) :

1. les zéros de tête de chaque groupe s'omettent : `0db8` → `db8` ;
2. **une seule** suite de groupes nuls se remplace par `::` :

```
2001:db8::42:8329
```

Le préfixe s'écrit comme en IPv4 : `2001:db8:abcd::/64` —

et `/64` est la taille standard d'un sous-réseau IPv6 (pas de pénurie, pas de calcul).

IPv6 : le minimum vital pour un admin cloud

À reconnaître au premier coup d'œil :

Adresse	Rôle	Équivalent IPv4
::1	loopback	127.0.0.1
fe80::/10	link-local (auto-configurée)	169.254.0.0/16
2000::/3	adresses publiques (GUA)	adresses publiques
fc00::/7	adresses privées (ULA)	RFC 1918
::/0	« tout Internet »	0.0.0.0/0

Points de repère cloud :

- pas de NAT en IPv6 : chaque machine peut avoir une adresse publique → le **pare-feu** redevient l'unique barrière ;
- AWS : un VPC peut être **dual-stack** (IPv4 + IPv6) ; les sous-réseaux IPv6 sont des /64 attribués – **aucun calcul à faire**, c'est prévu ainsi.

C'est tout pour ce bloc : IPv6 reviendra ponctuellement en CL-AWS1 et CL-SECU.

Récap des points clés du jour

- Le cloud est du **réseau défini par logiciel** : les concepts d'aujourd'hui sont les briques des VPC de vendredi.
- Modèles : **2 = MAC, 3 = IP, 4 = ports, 7 = application** ; encapsulation : **segment** → **paquet** → **trame**.
- ARP fait le pont IP → MAC **sur le réseau local uniquement**.
- Une adresse IPv4 = **32 bits**, le **masque** sépare réseau et hôte, le **CIDR** note le masque en **/n**.
- La méthode : **octet intéressant** → **taille de bloc (256 – masque)** → **réseau (multiple ≤)** → **broadcast (suivant – 1)**.
- **Hôtes** = $2^{(\text{bits hôte})} - 2$; plages privées : 10/8, 172.16/12, 192.168/16.
- IPv6 : 128 bits, notation **::**, sous-réseaux /64, **pas de calcul**.



Quiz — questions 1 et 2

Question 1

Combien de bits compte une adresse IPv4, et combien d'octets ?

Question 2

À quelle couche du modèle OSI correspond le protocole IP, et comment s'appelle l'unité de données (PDU) de cette couche ?



Quiz — questions 3 et 4

Question 3

Quel protocole permet de trouver l'adresse MAC correspondant à une adresse IP sur le réseau local, et par quel type de trame commence-t-il ?

Question 4

Quel est le masque en notation décimale pointée d'un réseau en `/27` ?



Quiz — questions 5 et 6

Question 5

Combien d'hôtes utilisables dans un sous-réseau `/26` ? Justifiez avec la formule.

Question 6

Quelle est l'adresse **réseau** du sous-réseau auquel appartient `10.0.0.130/25` ?



Quiz — questions 7 et 8

Question 7

Quelle est l'adresse de **broadcast** du réseau `192.168.4.0/23` ?

(Attention : la frontière n'est pas dans le 4e octet.)

Question 8

L'adresse `172.20.1.1` est-elle une adresse privée ? Et `172.32.1.1` ? Justifiez.



Quiz — questions 9 et 10

Question 9

Vous devez adresser un sous-réseau de **60 machines**. Quel est le préfixe le plus petit possible (celui qui gaspille le moins d'adresses) ?

Question 10

`2001:db8:0:0:0:0:1` — écrivez cette adresse IPv6 sous sa forme abrégée.

De combien de bits est constituée une adresse IPv6 ?



Jour 2 — Acheminer : routage, NAT, DHCP et DNS

Vos paquets savent maintenant **qui** ils cherchent.
Demain, ils apprennent **comment y aller** : tables de routage, passerelles, NAT (le grand tour de passe-passe d'Internet... et du cloud), et DNS en profondeur avec **dig**. Révissez la taille de bloc : on recommence l'échauffement à 9h05.